



O T U S

Онлайн образование

otus.ru

 Проверить, идет ли запись

Меня хорошо видно && слышно?



Базовый синтаксис Rust



Федченко Кирилл

Старший разработчик
НИИ Квант

`f3.otus@yandex.ru`



Правила вебинара



Активно участвуем



Задаем вопросы в чат



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Маршрут вебинара

Встроенные типы данных

Переменные, функции и
владение

Ветвление и циклы

Структуры, перечисления и
сопоставление с шаблоном

Комментарии

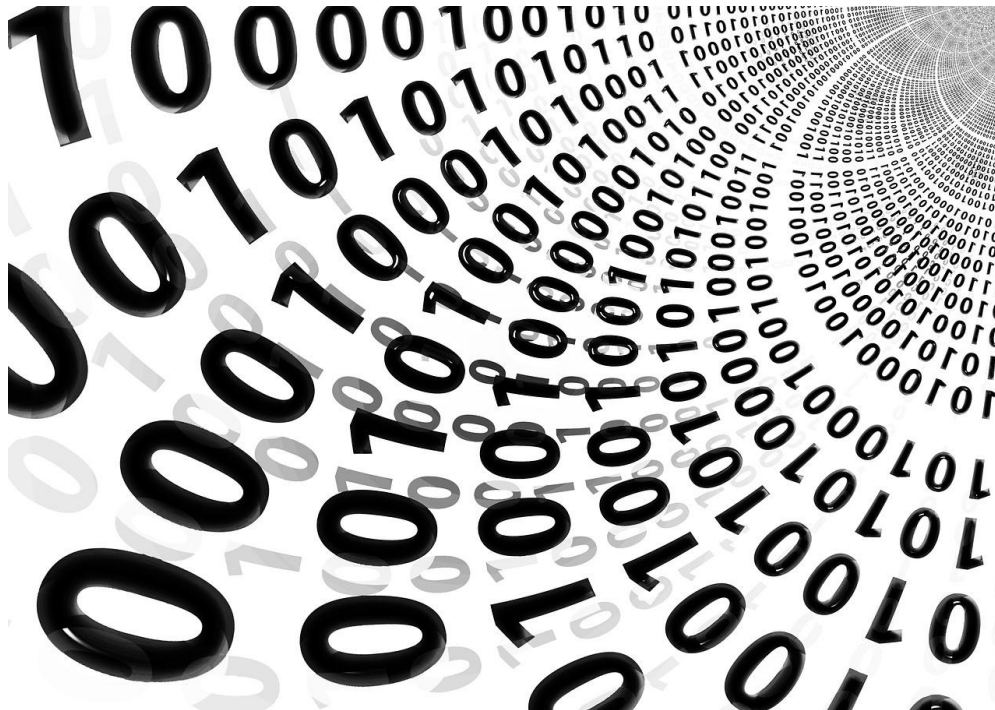


Встроенные типы данных



Встроенные типы данных

- 1) Скалярные
- 2) Составные
- 3) Слайсы



Скалярные типы

- 1) Булевый: **bool**
- 2) Целочисленные:
 - a) знаковые: **i8, i16, i32, i64, i128, isize**
 - b) беззнаковые: **u8, u16, u32, u64, u128, usize**
- 3) С плавающей точкой: **f32, f64**
- 4) Символ: **char**

О целых числах

- 1) Диапазоны
 - a) uX : $[0, 2^X - 1]$
 - b) iX : $[-2^{(X-1)}, 2^{(X-1)} - 1]$
- 2) Операции:
 - a) Арифметические: $+$, $-$, $*$, $/$, $\%$
 - b) Двоичные: $\wedge$, $|$, $\&$, $\ll$, $\gg$
 - c) Сравнение: $==$, $!=$, $<$, $>$, $\leq$, $\geq$
- 3) Неявное переполнение == ошибка
 $wrapping_*$ -> $i32$,
 $checked_*$ -> $Option<i32>$,
 $overflowing_*$ -> $(i32, bool)$,
 $saturating_*$ -> $i32$

Составные типы

- 1) Кортеж (tuple): `(i32, f64, bool)`
- 2) Массив (array): `[1, 2, 3, 3, 2]`

```
fn foo() {  
    let a : (i32, f64, bool) = (42, 4.2, true);  
    let b : [i32; 5] = [1, 2, 3, 3, 2];  
}
```

Переменные, функции и владение

Переменные

Объявляются:

```
let a : i32 = 42;
```

Переобъявляются:

```
let a : i32 = 42;  
let a : f64 = 4.2;
```

Изменяются:

```
let mut b : f64 = 4.2;  
b = b + 1;  
b -= 1;
```

Передаются:

```
let a : i32 = 42;  
let b = foo(a);
```

Static и const

Объявляются:

```
const SECS_IN_HOUR: u32 = 60 * 60;  
static GLOBAL_COUNTER: AtomicU64 = AtomicU64::new(v: 0);
```

Используются:

```
fn foo() {  
    let two_hours :u32 = SECS_IN_HOUR * 2;  
    let next_id :u64 = GLOBAL_COUNTER.fetch_add( val: 1,  
}
```

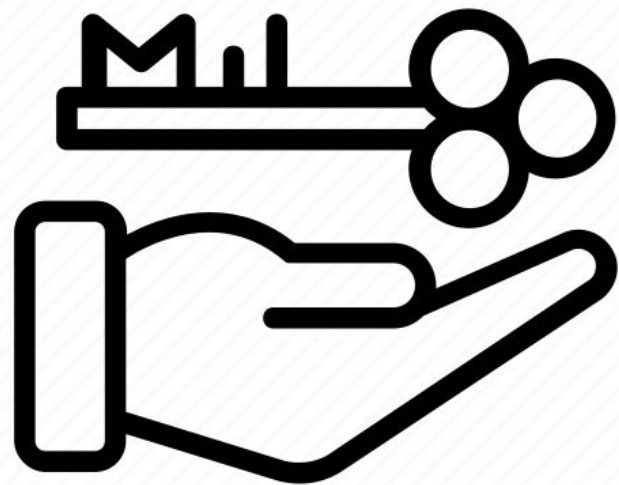
Функции

- Принимают аргументы и возвращают значение.
- Содержат выражения (expression) и утверждения (statement).

```
fn foo(int_arg: i32, float_arg: f64) → bool {  
    let int_as_f64 : f64 = int_arg as f64;  
    int_as_f64 > float_arg  
}
```

Владение

- Любые данные имеют переменную - владельца.
- В один момент времени у данных может быть только один владелец.
- Когда владелец выходит из области видимости, данные удаляются.



Владение

```
fn foo() {  
    let a : String = String::from(s: "Hello, world");  
    let b : String = a;  
    println!("{}", a); // Ошибка компиляции!  
  
    bar(_s: b.clone());  
    println!("{}", b); // "Hello, world"  
}  
  
fn bar(_s: String) {}
```


Ветвление и циклы

Циклы

```
for _i in 0..15 {}

let arr : [i32; 20] = [42; 20];
for _item in arr {}

while rand() > 0.5 {
    break
}
```

```
let _a = loop {
    break 3;
};

'outer: loop {
    while rand() > 0.5 {
        break 'outer;
    }
}
```

Ветвление

```
let number : i32 = 6;

if number % 4 == 0 {
    println!("делится на 4");
} else if number % 3 == 0 {
    println!("делится на 3");
} else if number % 2 == 0 {
    println!("делится на 2");
} else {
    println!("не делится");
}
```

```
let number : i32 = 6;

let _divisor = if number % 4 == 0 {
    4
} else if number % 3 == 0 {
    3
} else if number % 2 == 0 {
    2
} else {
    1
};
```

Структуры, перечисления и сопоставление с шаблоном

Структуры

```
struct Document {  
    name: String,  
    text: String,  
    views: u32,  
}  
  
fn main() {  
    let doc = Document {  
        name: String::from(s: "Some name"),  
        text: String::from(s: "Some text"),  
        views: 0  
    };  
}
```

Структуры

```
struct Document {  
    name: String,  
    text: String,  
    views: u32,  
}  
  
fn init_doc(name: String, text: String) → Document {  
    Document {  
        name,  
        text,  
        views: 0  
    }  
}
```

Структуры

```
let name : String = String::from(s: "Name");  
let text : String = String::from(s: "Text");  
let doc1 : Document = init_doc(name, text);  
let doc2 = Document {  
    name: String::from(s: "Other name"),  
    ..doc1  
};
```

Структуры - кортежи

```
struct Color(i32, i32, i32);  
struct Point(i32, i32, i32);  
  
fn main() {  
    let black = Color(i32: 0, i32: 0, i32: 0);  
    let origin = Point(i32: 0, i32: 0, i32: 0);  
    let g : i32 = black.1;  
    let z : i32 = origin.2;  
}
```


Перечисления

```
enum LoadError {  
    NotFound,  
    BufferTooSmall { actual: usize, expected: usize },  
    InternalError(String),  
}  
  
fn main() {  
    let err = LoadError::NotFound;  
    let err = LoadError::BufferTooSmall {  
        actual: 32,  
        expected: 64,  
    };  
    let msg : String = String::from(s: "Connection failed");  
    let err = LoadError::InternalError(msg);  
}
```

Перечисления

```
enum LoadError {  
    NotFound,  
    BufferTooSmall { actual: usize, expected: usize },  
    InternalError(String),  
}  
  
fn main() {  
    let err = LoadError::NotFound;  
    match err {  
        LoadError::NotFound => {}  
        LoadError::BufferTooSmall { expected: usize, .. } => {}  
        LoadError::InternalError(s : String ) => {}  
    }  
}
```

if let, while let

```
let msg : String = String::from(s: "connection failed");
let mut err = LoadError::InternalError(msg);
if let LoadError::InternalError(s : String) = err {
    println!("internal error: {}", s)
}
while let LoadError::InternalError(_) = err {
    err = try_load_again();
}
```

match

```
let x : i32 = 15;  
let s : String = match (x % 3, x % 5) {  
    (0, 0) => String::from(s: "FizzBuzz"),  
    (0, _) => String::from(s: "Fizz"),  
    (_, 0) => String::from(s: "Buzz"),  
    _ => x.to_string(),  
};
```

Сопоставление с шаблоном

```
struct Socket {  
    ip: u32,  
    port: u16,  
}  
  
fn main() {  
    let (a : i32 , b : f64 , c : &str ) = (1, 2., "hello");  
    let socket = Socket {  
        ip: 0xFF00FF00,  
        port: 80,  
    };  
    let Socket { ip : u32 , .. } = socket;  
}
```

Сопоставление с шаблоном

```
fn foo((a, b): (i32, f32)) {}
```

```
struct Socket {
```

```
    ip: u32,
```

```
    port: u16,
```

```
}
```

```
fn bar(Socket { ip, .. }: Socket) {}
```

impl блоки

```
struct User {  
    name: String,  
    age: u32,  
}  
  
impl User {  
    fn new() → Self {  
        let name: String = String::from("Alice");  
        User { name, age: 42 }  
    }  
    fn get_age(&self) → u32 {  
        self.age  
    }  
}  
  
fn foo() {  
    let user: User = User::new();  
    println!("User age: {}", user.age);  
}
```


ітрі блоки

```
enum Address {
    IPv4([u8; 4]),
    IPv6([u8; 16]),
}

impl Address {
    pub fn v4(addr: [u8; 4]) → Self {
        Self::IPv4(addr)
    }

    pub fn v6(addr: [u8; 16]) → Self {
        Self::IPv6(addr)
    }

    fn print(&self) {
        match self {
            Address::IPv4(addr: &[u8; 4]) ⇒ println!("v4: {addr:?}"),
            Address::IPv6(addr: &[u8; 16]) ⇒ println!("v6: {addr:?}"),
        }
    }
}

fn bar() {
    let addr: Address = Address::v4([127, 0, 0, 1]);
    addr.print();
}
```


Комментарии

Обычные

```
// деконструкция кортежа
let (a : i32, b : f64, c : &str) = (1, 2., "hello");

let socket = Socket { // создаём объект Socket
    ip: 0xFF00FF00, // ip адрес
    port: 80,
};

/* комментарий
   на две строки */
let Socket { ip : u32, .. } = socket;
```

Документационные

```
/// Adds one to the number given.
///
/// # Examples
///
/// ```
/// let arg = 5;
/// let answer = my_crate::add_one(arg);
///
/// assert_eq!(6, answer);
/// ```
pub fn add_one(x: i32) → i32 {
    x + 1
}
```

Документационные

my_crate

Functions

add_one

Crates

my_crate



Click or press 'S' to search, '?' for more options...



Function my_crate::add_one

[–][src]

```
pub fn add_one(x: i32) -> i32
```

[–] Adds one to the number given.

Examples

```
let arg = 5;  
let answer = my_crate::add_one(arg);  
  
assert_eq!(6, answer);
```

Документационные

```
add_one( x: 5);
```

```
dz_temp
```

```
pub fn add_one(x: i32) -> i32
```

Adds one to the number given.

Examples

```
let arg = 5;
```

```
let answer = my_crate::add_one(arg);
```

```
assert_eq!(6, answer);
```

⋮

Вопросы?

Спасибо за внимание!